

Using JSON Web Tokens (JWTs)



Scott Brady

Identity Consultant & Engineering Manager

@scottbrady91

www.scottbrady91.com



Overview



How JWTs are created

How to securely validate JWTs and when to reject them

How to take advantage of standard claims types

JWTs in action:

- OpenID Connect & OAuth 2
- Online tools
- JWT developer libraries



How to Create a JWT



Digital Signatures



JWT Header

Describes the token format and how it has been cryptographically secured.



```
{  
  "typ": "JWT"  
  
}
```

Token Type (typ)

The token's media type.

Optional.

```
{  
  "typ": "application/json"  
}
```

Token Type (typ)

The token's media type.

Optional.

```
{  
  "typ": "JWT"  
  
}
```

Token Type (typ)

The token's media type.

Optional.

```
{  
  "typ": "JWT",  
  "alg": "RS256"  
  
}
```

Algorithm (alg)

The algorithm used to sign the token.

Required.

```
{  
  "typ": "JWT",  
  "alg": "RS256",  
  "kid": "f7811b04be0844c0b8c77250504623d9"  
}
```

Key ID (kid)

Identifies the public key that should be used to verify the token.

Optional.

JWT Payload

Claims about an entity – the contents of the token.



```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com"  
  
}
```

Issuer (iss) and Audience (aud)

Who created the token & who the token should be read by.

Useful when you have many token issuers or APIs.

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314  
  
}
```

Expires At (exp)

When the token expires.

Unix time (seconds since the epoch).

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "iat": 1656060014  
}
```

Issued At (exp)

When the token was created.

Unix time (seconds since the epoch).

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "iat": 1656060014,  
  "sub": "b3bfa8d95a17468292a87995e36552dd"  
}
```

Subject (sub)

Who the token represents (user or machine).

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "iat": 1656060014,  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "name": "Scott Brady"  
}
```

Anything Else

Claims that might be useful.

E.g. basic identity or protocol-specific data.

Base64 vs Base64url Encoding

Base64

A-Za-z0-9+/=

Includes padding

Base64url

A-Za-z0-9-

Doesn't include padding



JWT Header Encoding

Header

```
{  
  "typ": "JWT",  
  "alg": "RS256",  
  "kid": "f7811b04be0844c0b8c77250504623d9"  
}
```

`base64url(utf8($header))`

Encoded header

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODExYj04YmEwODQ0YzBiOGM3NzI1MDUwNDYyM2Q5In0
```



JWT Header Encoding

Header

```
{ "typ": "JWT", "alg": "RS256", "kid": "f7811b04b  
e0844c0b8c77250504623d9" }
```

base64url(utf8(\$header))

Encoded header

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODQ4c0b8c77250504623d9In0
```



JWT Payload Encoding

• Payload

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "name": "Scott Brady"  
}
```

base64url(utf8(\$payload))

• Encoded payload

```
eyJpc3MiOiJodHRwczovL2lkC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwiaXhwIjoxNjU2MDYwMzE0LCJzdWIiOiJiM2JmYThkOTVhMTc0NjgyOTJhODc5OTVlMzY1NTJkZCIsIm5hbWUiOiJTY290dCBCcmFkeSJ9
```



Signature

Encoded header + payload

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODExYjA0YmUwODQ0YzBiOGM3NzI1MDUwNDYyM2Q5In0.eyJpc3MiOiJodHRwczovL2lkcC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwiaXhwIjozNjU2MDYwMzE0LCJzdWIiOiJmM2JmYTlkOTVhMTc0NjgyOTJhODc5OTVlMzY1NTJkZCIsIm5hbWUiOiJTY290dCBCcmFkeSJ9
```

base64url(sign(ascii(\$encodedData)))

Encoded signature

```
pu7BRJUrnCeGo3hcuk6jvDVG19Hvf5r3Cinnl0NjpiuZqG1-ijLCqnFHPXpVq-M4BqQzLv7tity27GerMtIt3A3P2sHiIo9jF7iN7Q9fu8nlt3HpKa50ypHlb44Iqad-SzM3seapGhl9ruuDGBTv6ZSoaPUq6BbWE5gNVW6N1r-OTrgargdcM1MHeeqRp0c13Uai84Zn7XbBiz2rFQJhRmvGnOTx9ZwqW-TCz4IWGWx2AtmxQYI76U6axyg0ownqI648oBB3NfZK-GrwE17vK_DZwUcrEJdsd0hFxNHSRjcFMP1bxtIXFiXTQ0yggqRHwD5BPyoEnyeyNSRh-iTCawh-02SKl1XRciS123V1RHXWduMJG9sVBT3-caJHgNIJQFrJ8Nq5SmIVEuTldPVW7YC--hV6CW-LbWbz1YfnVQwhYQbL3w0166aPYHqsLyJ70II4YytBRvvoyjBV1rkIbY14Xsc5G9-0CNttyh3h6jlRBGnpNFL0okLz_cGy9Fzs
```



JSON Web Token

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODExYjA0YmUwODQ0YzBiOGM3NzI1MDUwNDYyM2Q5In0.eyJpc3MiOiJodHRwczovL2lkZC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwiaXhwIjojNjU2MDYwMzE0LCJzdWIiOiJiM2JmYTlkOTVhMTc0NjgyOTJhODc5OTVlMzY1NTJkZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwiaWF0IjoxNjU2MDYwMzE0LCJpdjpuZqGl-ijLCqnFHPXpVq-M4BqQzLv7tity27GerMtIt3A3P2sHiIo9jF7iN7Q9fu8nlt3HpKa50ypHlb44Iqad-SzM3seapGhl9ruuDGBTv6ZSoaPUq6BbWE5gNVW6N1r-OT-rgargdcM1MHeeqRp0c13Uai84Zn7XbBiz2rFQJhRmvGnOTx9ZwqW-TCz4IWGWx2AtmxQYI76U6axyg0ownqI648oBB3NfZK-GrwE17vK_DZwUcrEJdsd0hFxNHSRjcFMP1bxtIXFiXTQ0yggqRHwD5BPyoEnyeyNSRh-iTCawh-02SKl1XRciS123VlRHXWduMJG9sVBT3-caJHgNIJQFrJ8Nq5SmIVEuTldPVW7YC--hV6CW-LbWbz1YfnVQwhYQbL3w0166aPYHqsLyJ70II4YytBRvvoyjBV1rkIbYl4Xsc5G9-0CNttyh3h6jlRBGnpNFL0okLz_cGy9Fzs
```



How to Validate a JWT



Verify Format

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODExYjA0YmUwODQ0YzBiOGM3NzI1MDUwN
DYyM2Q5In0.eyJpc3MiOiJodHRwczovL2lkcC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLm
V4YW1wbGUuY29tIiwiaXhwIjozNjU2MDYwMzE0LCJzdWIiOiJiM2JmYTlkOTVhMTc0NjgyOTJhODc5OTV
lMzY1NTJkZCI6Im5hbWUiOiJTY290dCBCcmFkeSJ9. pu7BRJUrnCeGo3hcuk6jvDVG19Hvf5r3Cinn10N
jpiuZqGl-ijLCqnFHPXpVq-M4BqQzLv7tity27GerMtIt3A3P2sHiIo9jF7iN7Q9fu8n1t3HpKa50ypHl
b44Iqad-SzM3seapGh19ruuDGBTv6ZSoaPUq6BbWE5gNVW6N1r-0T-rgargdcM1MHeeqRp0c13Uai84Zn
7XbBiz2rFQJhRmvGn0Tx9ZwqW-TCz4IWGWx2AtmxQYI76U6axyg0ownqI648oBB3NfZK-GrwE17vK_DZw
UcrEJdsd0hFxNHSRjcFMP1bxtIXFiXTQ0ygqRHwD5BPyoEnyeyNSRh-iTCawh-02SK11XRciS123V1RHX
WduMJG9sVBT3-caJHgNIJQFrJ8Nq5SmIVEuTldPVW7YC--hV6CW-LbWbz1YfnVQwhYQbL3w0166aPYHqs
LyJ70II4YytBRvvoyjBV1rkIbY14Xsc5G9-0CNttyh3h6jlRBGnpNFL0okLz_cGy9Fzs



Initial Format Checks

Three sections?

Two dots?

Base64url data?

Valid JSON objects?



```
{  
  "typ": "JWT",  
  "alg": "RS256",  
  "kid": "f7811b04be0844c0b8c77250504623d9"  
}
```

Header Validation

Must only contain values you understand.

Header Validation

**Signed using a key
you know?**

**Should this key be
used for this
algorithm?**

**Does the key
belong to this
issuer?**



Payload Validation

**Is the token issued
by someone you
know?**

**Are you the
intended recipient
of the token?**

**Has the token
expired?**



Signature Validation

• **Encoded header + payload**

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtpZCI6ImY3ODExYjA0YmUwODQ0YzBiOGM3NzI1MDUwNDYyM2Q5In0.eyJpc3MiOiJodHRwczovL2lkY21kZC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwiaXhwIjojoxNjU2MDYwMzE0LCJzdWIiOiJiM2JmYTlkOTVhMTc0NjgyOTJhODc5OTVlMzY1NTJkZCI6Im5hbWUiOiJTY290dCBCcmFkeSJ9

• **Encoded signature**

pu7BRJUrnCeGo3hcuk6jvDVG19Hvf5r3Cinnl0NjpiuZqG1-ijLCqnFHPXpVq-M4BqQzLv7tity27GerMtIt3A3P2sHiIo9jF7iN7Q9fu8nlt3HpKa50ypH1b44Iqad-SzM3seapGh19ruuDGBTv6ZSoaPUq6BbWE5gNVW6N1r-OTrgargdcM1MHeeqRp0c13Uai84Zn7XbBiz2rFQJhRmvGnOTx9ZwqW-TCz4IWGWx2AtmxQYI76U6axyg0ownqI648oBB3NfZK-GrwE17vK_DZwUcrEJdsd0hFxnHSRjcFMP1bxtIXFiXTQ0ygqRHwD5BPyoEnyeyNSRh-iTCawh-02SKl1XRciS123V1RHXWduMJG9sVBT3-caJHgNIJQFrJ8Nq5SmIVEuTldPVW7YC--hV6CW-LbWbz1YfnVQwhYQbL3w0166aPYHqsLyJ70II4YytBRvvoyjBV1rkIbY14Xsc5G9-0CNttyh3h6j1rBGnpNFL0okLz_cGy9Fzs



Public key



Valid?



Always validate a JWT.



```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "name": "Scott Brady"  
}
```

← Can this user access this API?

Custom Payload Validation

Validate request itself.

Validate user permissions.

Using Standard Claim Types for Common JWT Functionality



Use the standard claims in order to use JWTs securely.



```
"iss": "me"
```

```
"iss": "urn:pluralsight:idp"
```

```
"iss": "https://identity.pluralsight.com"
```

Issuer

Who created the token.

String or URI.

Issuer Validation

**Do you know this
token issuer?**

**Was the signature
created using a
key associated
with this issuer?**

**Does the subject
belong to this
issuer?**



```
"aud": "api1"
```

```
"aud": "urn:pluralsight:api1"
```

```
"aud": "https://api1.pluralsight.com"
```

Audience

The intended recipient of the token.

String or URI.

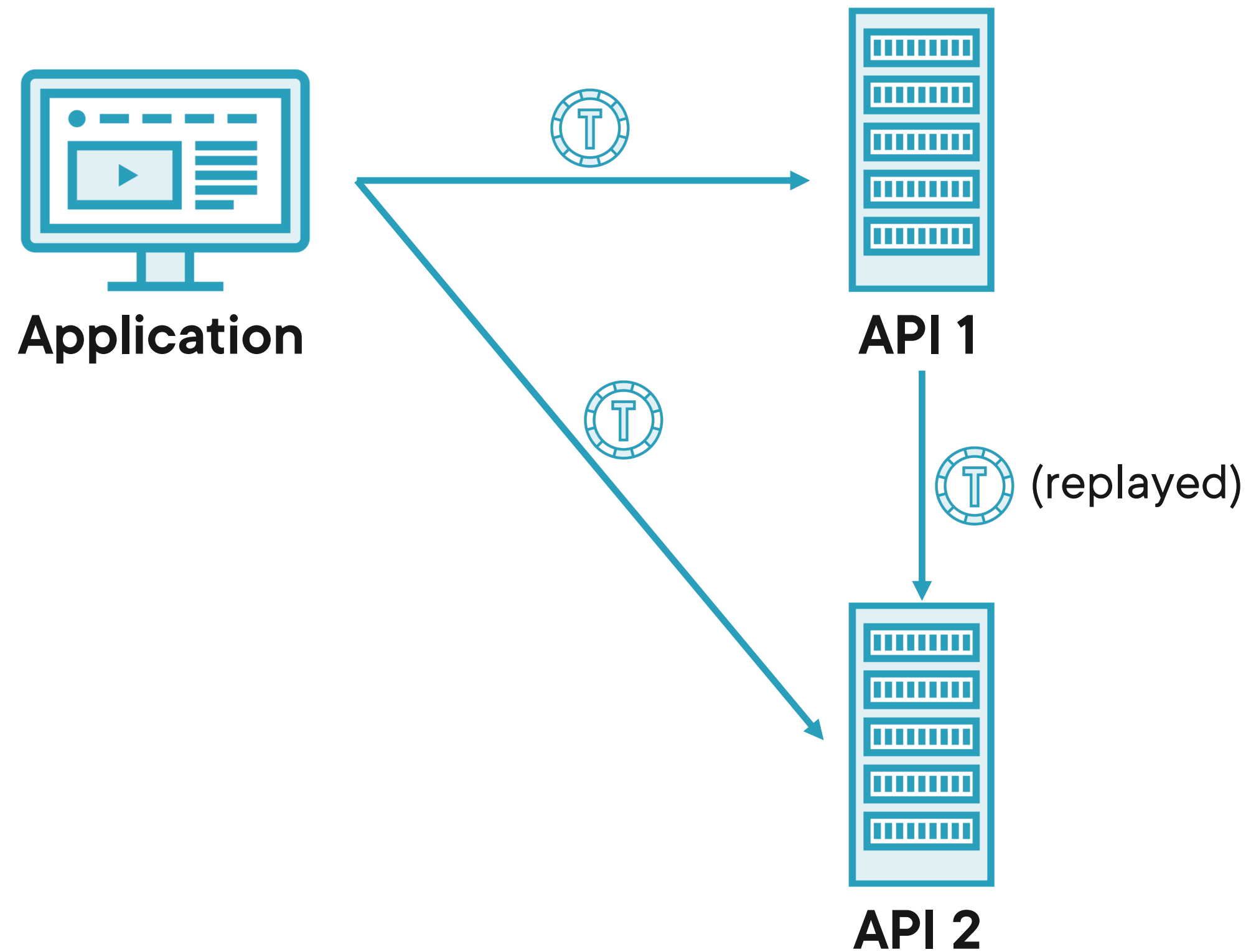
```
"aud" : [  
  "api1",  
  "api2"  
]
```

Audience(s)

The intended recipients of the token.

Be aware of token replay.

Token Replay



```
"sub" : "b3bfa8d95a17468292a87995e36552dd"
```

```
"sub" : "scott@scottbrady91.com"
```

Subject

Unique identifier of user or system.

Payload contains claims about the subject.

```
"exp" : 1656060314
```

Expires At

When the token expires.

Unix time (seconds since the epoch).

Consider clock skew.

```
"iat": 1656060014
```

Issued At

When the token was created.

Unix time (seconds since the epoch).

Useful for custom token lifetime checks.

```
"nbf" : 1656060014
```

Not Before

If in the future, the token is not valid.

```
"jti": "3069836a0df849b49c9b24da977971a3"
```

JWT ID

The unique identifier of this token.

Provide a mechanism for blocklists or one-time use tokens.

Registered Claims

Claim Name ⓘ	Claim Description ⓘ	Change Controller ⓘ	Reference ⓘ
iss	Issuer	[IESG]	[RFC7519, Section 4.1.1]
sub	Subject	[IESG]	[RFC7519, Section 4.1.2]
aud	Audience	[IESG]	[RFC7519, Section 4.1.3]
exp	Expiration Time	[IESG]	[RFC7519, Section 4.1.4]
nbf	Not Before	[IESG]	[RFC7519, Section 4.1.5]
iat	Issued At	[IESG]	[RFC7519, Section 4.1.6]
jti	JWT ID	[IESG]	[RFC7519, Section 4.1.7]
name	Full name	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
given_name	Given name(s) or first name(s)	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
family_name	Surname(s) or last name(s)	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
middle_name	Middle name(s)	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
nickname	Casual name	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
preferred_username	Shorthand name by which the End-User wishes to be referred to	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
profile	Profile page URL	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
picture	Profile picture URL	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
website	Web page or blog URL	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
email	Preferred e-mail address	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
email_verified	True if the e-mail address has been verified; otherwise false	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
gender	Gender	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
birthdate	Birthday	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
zoneinfo	Time zone	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]
locale	Locale	[OpenID Foundation Artifact Binding Working Group]	[OpenID Connect Core 1.0, Section 5.1]

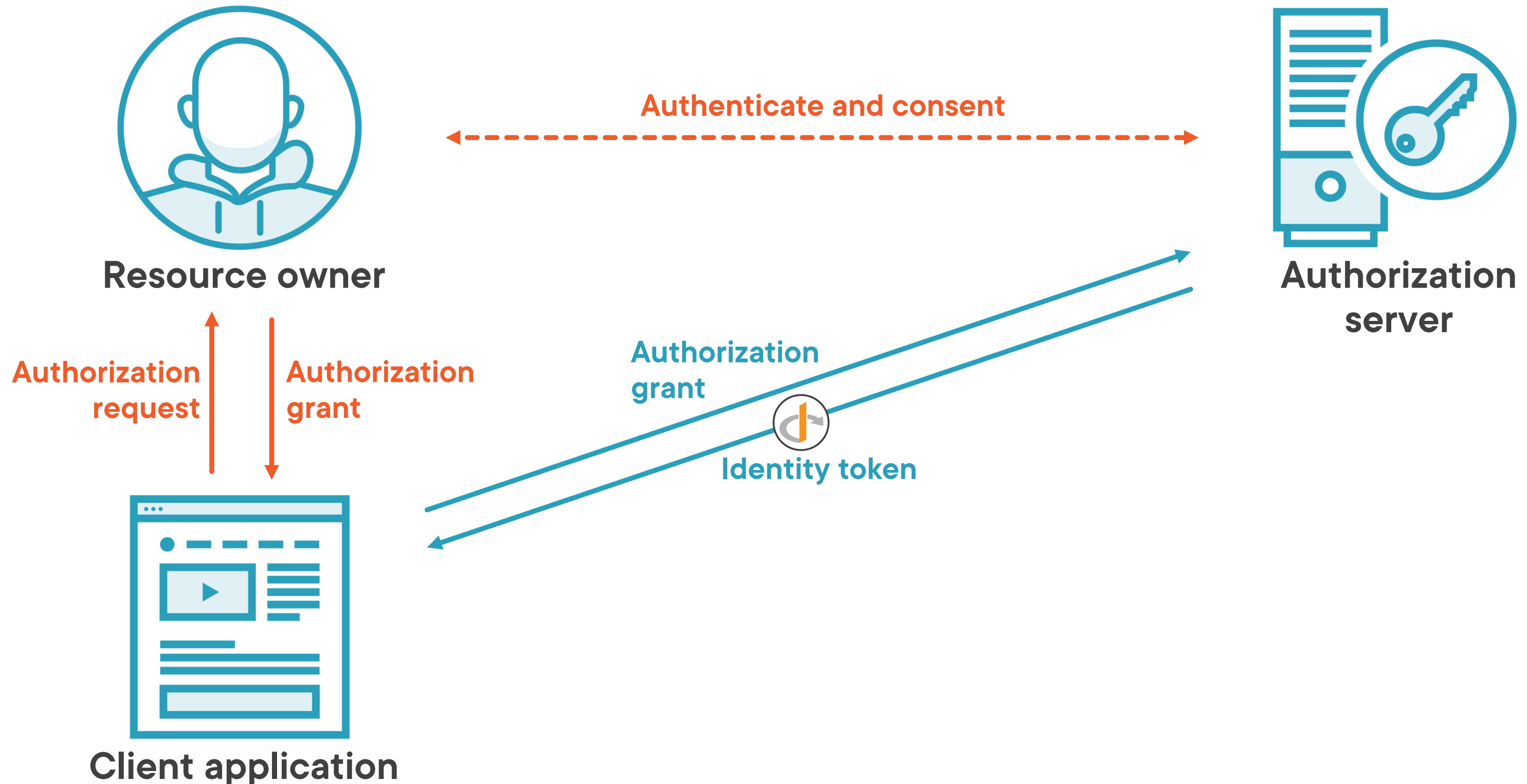
<https://www.iana.org/assignments/jwt/jwt.xhtml#claims>



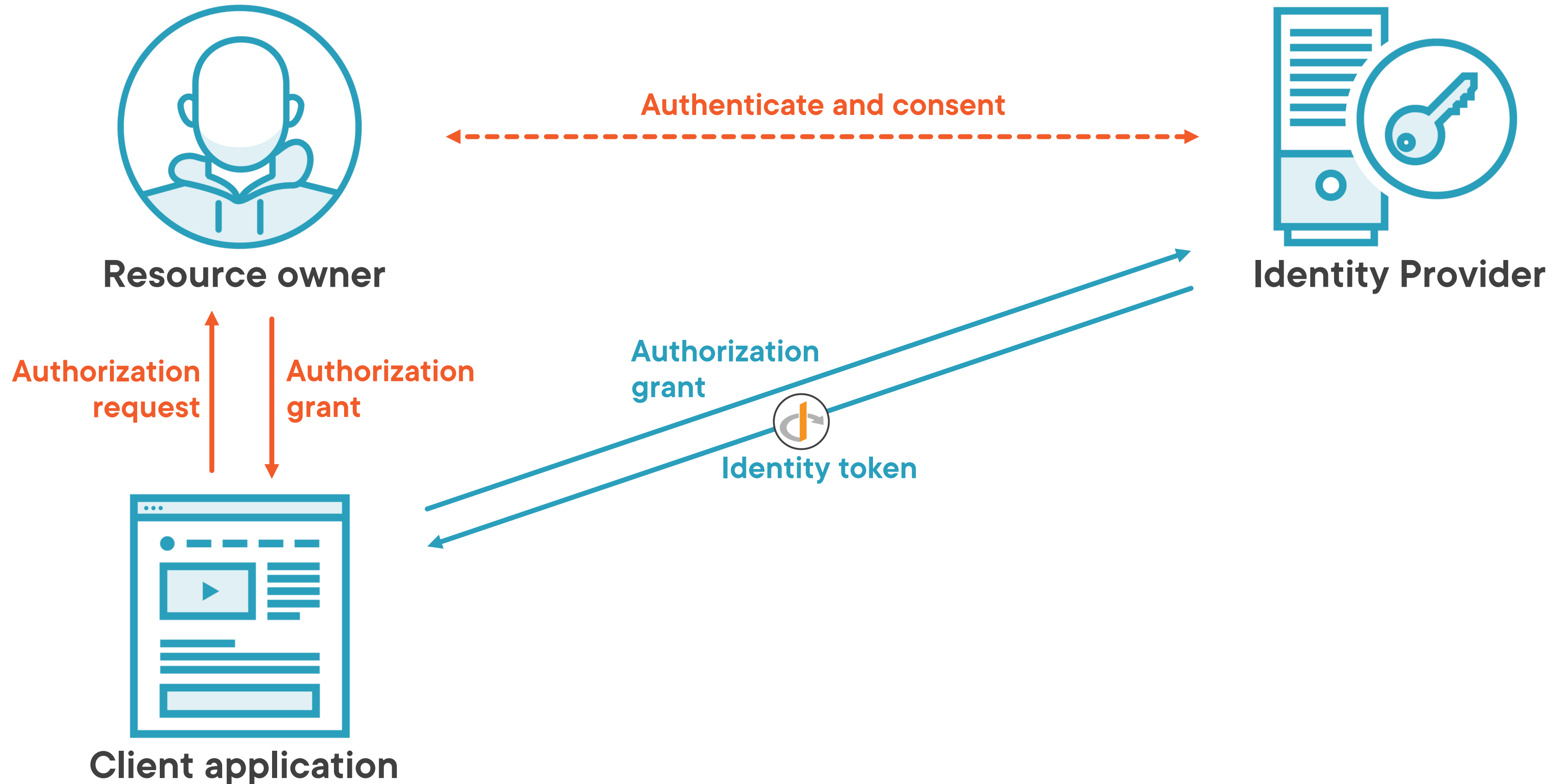
JWTs in Use: OpenID Connect's Identity Token



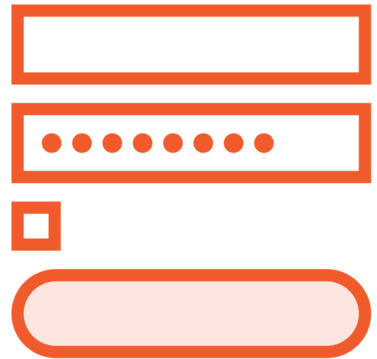
The OpenID Connect Dance



The OpenID Connect Dance



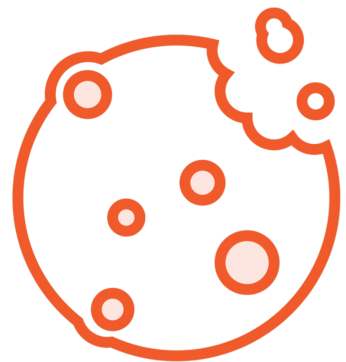
Identity Tokens



Describe the authentication event (how the user authenticated)



Intended for the client application



Client application uses the token to decide if it wants to start a session for that user



```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://client.example.com",  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "exp": 1602927777,  
  "iat": 1602927477  
}
```

- ◀ **Issuer must be an HTTPS URL**
- ◀ **Audience must be the `client_id` of the client app**
- ◀ **Subject must be the user who authenticated**
- ◀ **Short-lived**

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://client.example.com",  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "exp": 1602927777,  
  "iat": 1602927477,  
  "auth_time": 1602927277,  
  "amr": [ "pwd", "mfa" ],  
  "acr": "urn:openbanking:psd2:sca",  
  "nonce": "edf037015b41[... ]6e38a40e44912",  
  "sid": "472ae8c4776c4d109a9ed7ddb8780cfa"  
}
```

- ◀ **Issuer must be an HTTPS URL**
- ◀ **Audience must be the `client_id` of the client app**
- ◀ **Subject must be the user who authenticated**
- ◀ **Short-lived**
- ◀ **When the user last authenticated**
- ◀ **What methods the user used to authenticate**
- ◀ **What level the user authenticated at**
- ◀ **Random value from the authorization request, echoed back in the identity token**
- ◀ **Session ID**

The identity part of an identity token is optional!





More Information

Understanding Identity Tokens

Scott Brady



JWTs in Use: JWT Profile for OAuth Access Tokens



JWT Access Tokens

Different structure, same meaning

Okta

```
{
  "iss": "https://my-domain.okta.com/",
  "aud": "https://api.example.com",
  "sub": "00ujmkLgagxeRrAg20g3",
  "cid": "nmdP1fcyvdV011AL7ECm",
  "iat": 1467145094,
  "exp": 1467148694,
  "scp": [ "openid", "email", "flights" ],
  "ver": 1,
  "uid": "00ujmkLgagxeRrAg20g3",
  "auth_time": 1467142021
}
```

Auth0

```
{
  "iss": "https://my-domain.auth0.com/",
  "aud": [
    "https://example.com/health-api",
    "https://my-domain.auth0.com/userinfo"
  ],
  "sub": "auth0|123456",
  "azp": "my_client_id",
  "iat": 1311280970,
  "exp": 1311281970,
  "scope": "openid email read:patients"
}
```

RFC 9068

JWT Profile for OAuth 2.0 Access Tokens



```
{  
  "typ": "at+jwt",  
  "alg": "RS256",  
  "kid": "f7811b04be0844c0b8c77250504623d9"  
}
```

JWT Access Token Media Type

Media type “application/at+jwt”.

Must be validated when using this profile.

```
{  
  
  "iss": "https://idp.example.com",  
  
  "aud": "https://api.example.com",  
  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  
  "exp": 1656060614,  
  
  "iat": 1656060014,  
  
  "jti": "0edc56fd27f77f1e3fafb6df8cb99acd",  
  
  "client_id": "43862154c2b54cfaa2f1e92d96b9ad1a"  
  
}
```

Required JWT Payload

Subject and client ID claim can be equal (when no user is present).

JWT Access Token Profile

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api1.example.com",  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "exp": 1656060614,  
  "iat": 1656060014,  
  "jti": "0edc56fd27f77f1e3fafb6df8cb99acd",  
  "client_id": "43862154c2b54cfaa2f1e92d96b9ad1a",  
  "auth_time": 1602927277, ← OpenID Connect  
  "amr": [ "pwd", "mfa" ], ← OpenID Connect  
  "acr": "urn:openbanking:psd2:sca", ← OpenID Connect  
  "scope": "openid profile api1.read" ← Authorized OAuth scopes  
}
```



Validate the payload and reject
unsigned tokens!





More Information

Getting Started with OAuth 2

Scott Brady



Useful JWT Tools



JWT Parsing Tools

jwt.io

jwt.ms

[gchq.github.io
/CyberChef](https://gchq.github.io/CyberChef)



JWT Creation Tools

scottbrady91.com/tools/jwt



Do not upload private data to
online tools.



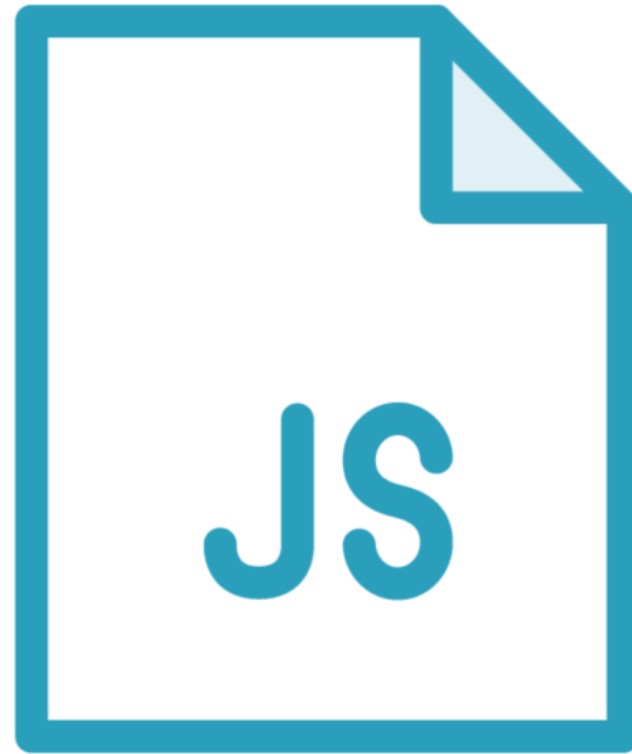
JWT Libraries



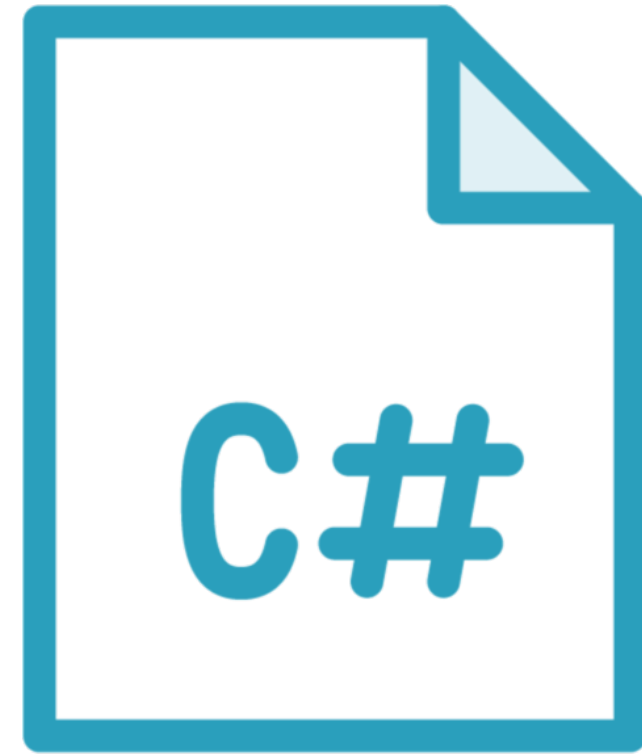
Demos



Python



JavaScript



C#



For demonstration purposes
only.



Choosing a JWT Library

JWT

DebuggerLibrariesIntroductionAsk

Crafted by  auth0 ?



Libraries for Token Signing/Verification

Filter by

All

All

.NET

1C

Ada

C

C++

CFML

Clojure

Crystal

D

Delphi

Deno

Elixir

Erlang

Go

Groovy

Harbour

Haskell

Haxe

Java

.NET		.NET		.NET	
✓ Sign	✓ HS256	✓ Sign	✓ HS256	✓ Sign	✓ HS256
✓ Verify	✓ HS384	✓ Verify	✓ HS384	✓ Verify	✓ HS384
✓ iss check	✓ HS512	✓ iss check	✓ HS512	✗ iss check	✓ HS512
✓ sub check	✓ PS256	✓ sub check	✗ PS256	✗ sub check	✓ PS256
✓ aud check	✓ PS384	✓ aud check	✗ PS384	✗ aud check	✓ PS384

<https://jwt.io/libraries>



Demo



JWT creation and validation in Python using Authlib



Demo



JWT creation and validation in JavaScript using jose



Demo



**JWT creation and validation in C# using
Microsoft.IdentityModel**



Summary



How JWTs are created

How to securely validate JWTs and when to reject them

How to take advantage of standard claims types

JWTs in action:

- OpenID Connect & OAuth 2
- Online tools
- JWT developer libraries



JSON Web Signatures (JWS) in Depth

